

PROGRAM CONTROL STATEMENTS

Basics about Arrays

- ✓ **An array** is an indexed group of data storage locations that have the same name and are distinguished from each other by an index--**a number following the variable name, enclosed in brackets.**
- ✓ Like other C variables, arrays must be declared.
- ✓ An array declaration includes both the data type and the size of the array (the number of elements in the array).
- ✓ For example, the statement below declares an array named data that is of type int and has 1,000 elements:
- ✓ **int data[1000];**
- ✓ The individual elements are referred to by index as data[0] through data[999].
- ✓ The first element is data[0], not data[1].

- ✓ Each element of this array is equivalent to a normal integer variable and can be used the same way.
- ✓ The index of an array can be another C variable, as in this example:

```
int data[1000];
```

```
int count;
```

```
count = 100;
```

```
data[count] = 12;    /* The same as data[100] = 12 */
```

- ✓ **The above means that the element at index 100 of the data array is 12**

Controlling Program Execution

- ✓ The default order of execution in a C program is top-down.
- ✓ Execution starts at the beginning of the `main()` function and progresses, statement by statement, until the end of `main()` is reached.
- ✓ However, this order is rarely encountered in real C programs.
- ✓ The C language includes a variety of program control statements that let you control the order of program execution.
- ✓ You have already learned how to use C's fundamental decision operator, the `if` statement.

The for Statement

- ✓ The for statement is a programming construct that executes a block of one or more statements a certain number of times.
- ✓ It is sometimes called the **for loop** *because program execution typically loops through* the statement more than once.
- ✓ A for statement has the following structure:

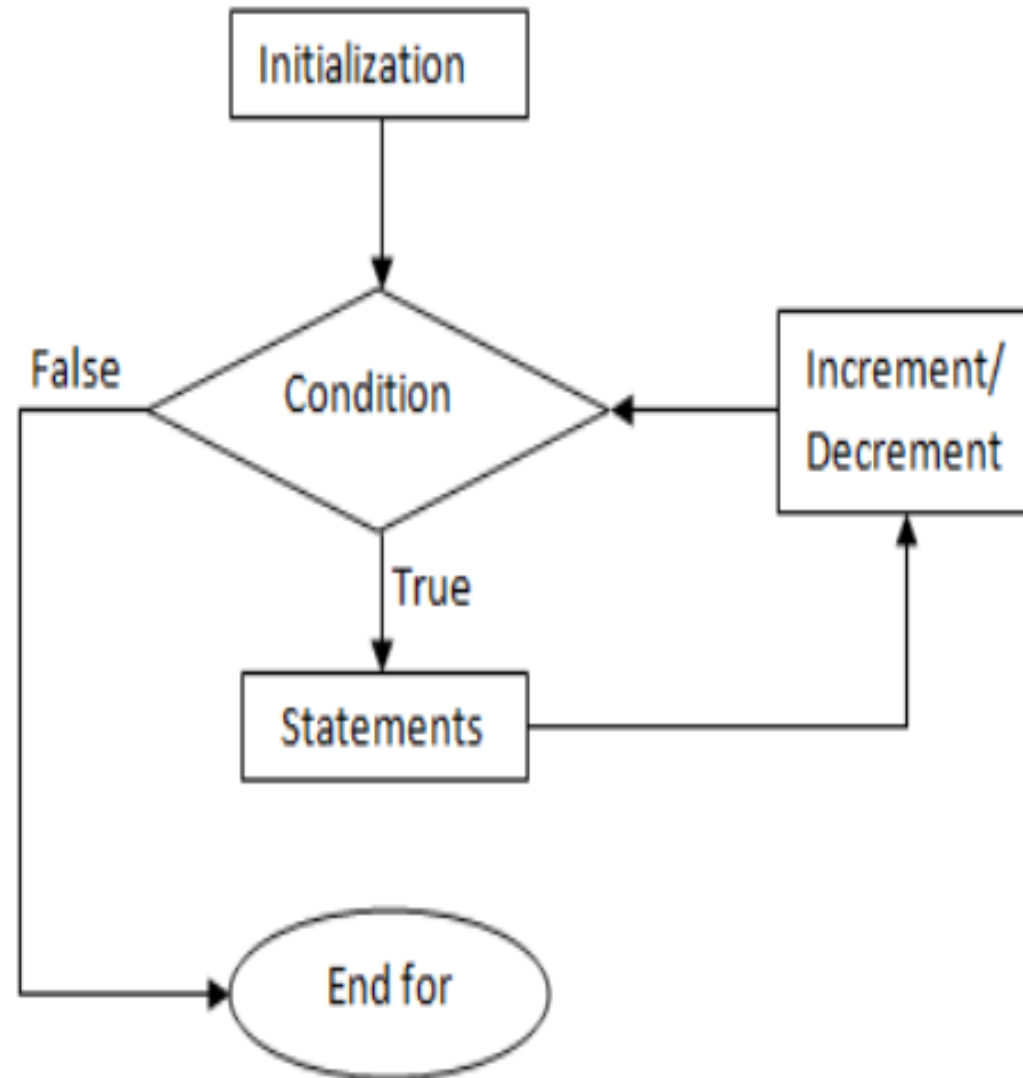
***for (initialization; condition; increment/decrement)
statement(s);***

- ✓ *initial, condition, and increment are all C expressions, and statement is a single or compound C statement.*

✓ When a for statement is encountered during program execution, the following events occur:

1. The expression **initial is evaluated**. initial is usually an assignment statement that sets a variable to a particular value (starting point).
2. The **expression condition** is checked. condition is typically a relational expression.
3. If condition evaluates to fals, the for statement terminates, and execution passes to the first statement next statement (**outside the for loop**).
4. If condition evaluates to true , the C statement(s) in the for loop are executed.
5. The expression **increment is executed**, and execution returns to step 2.

Flow Chart of the for loop



Example program of the for loop

1		<code>#include <stdio.h></code>
2		<code>int main()</code>
3	☐	<code>{</code>
4		
5		<code>int x;</code>
6		
7		<code>for(x=0;x<=10;x++)</code>
8	☐	<code>{</code>
9		<code>printf("%d\n",x);</code>
10		<code>}</code>
11		
12		<code>printf("The Program has ended");</code>
13		
14		<code>return 0;</code>
15		<code>}</code>
16		

- ✓ The for statement is quite flexible.
- ✓ For example, you can omit the initialization expression if the test variable has been initialized previously in your program.
- ✓ **Note:** (You still must use the semicolon separator as shown)

```
count = 1;
```

```
for ( ; count < 1000; count++)
```

- ✓ The initialization expression doesn't need to be an actual initialization; it can be any valid C expression.
- ✓ Whatever it is, it is executed once when the for statement is first reached.
- ✓ For example, the following prints the statement **Now sorting the array...**

```
count = 1;
```

```
for (printf("Now sorting the array...") ; count < 1000; count++)
```

- ✓ You can also omit the increment expression and perform the updating in the body of the for statement.
- ✓ Again, the semicolon must be included. To print the numbers from 0 to 99, you could write

```
for (count = 0; count < 100; )
```

```
    printf("%d", count++);
```

- ✓ You can also be able to initialize and increment/decrement two variables within the same for loop.

```
for (i = 0, j = 999; i < 1000; i++, j--)  
b[j] = a[i];
```

- ✓ The above example looks at two 1,000-element arrays, a[] and b[].
- ✓ The for loop is used to copy the contents of a[] to b[] in reverse order so that after the copy operation, b[0] = a[999], b[1] = a[998], and so on.
- ✓ The comma operator is used to initialize the two variables, i and j.
- ✓ It is also used to increment and decrement part of these two variables with each loop.

Nesting for Statements

- ✓ A for statement can be executed within another for statement. This is called *nesting*. (You saw this with the if statement.)
- ✓ By nesting for statements, you can do some complex programming.
- ✓ Listing 6.2 is not a complex program, but it illustrates the nesting of two for statements.

Example program of the Nested for loop

```
1  /* Demonstrates nesting two for statements */
2
3  #include <stdio.h>
4
5  void draw_box( int, int);
6
7  main()
8  {
9      draw_box( 8, 35 );
10
11      return 0;
12  }
13
14  void draw_box( int row, int column )
15  {
16      int col;
17      for ( ; row > 0; row--)
18      {
19          for (col = column; col > 0; col--)
20              printf("X");
21
22          printf("\n");
23      }
24
25  }
```

- ✓ The program on the previous slide prints 35 Xs horizontally and 8 Xs vertically creating a rectangle of 8 rows and 35 columns.
- ✓ This program uses both a function called **draw_box** and two nested for loops to achieve its purpose.
- ✓ Try to look at the program and discuss with your colleagues on how the program execution proceeds from the outer for loop to the inner for loop until the conditions for the two loops all become false.

The While Statement

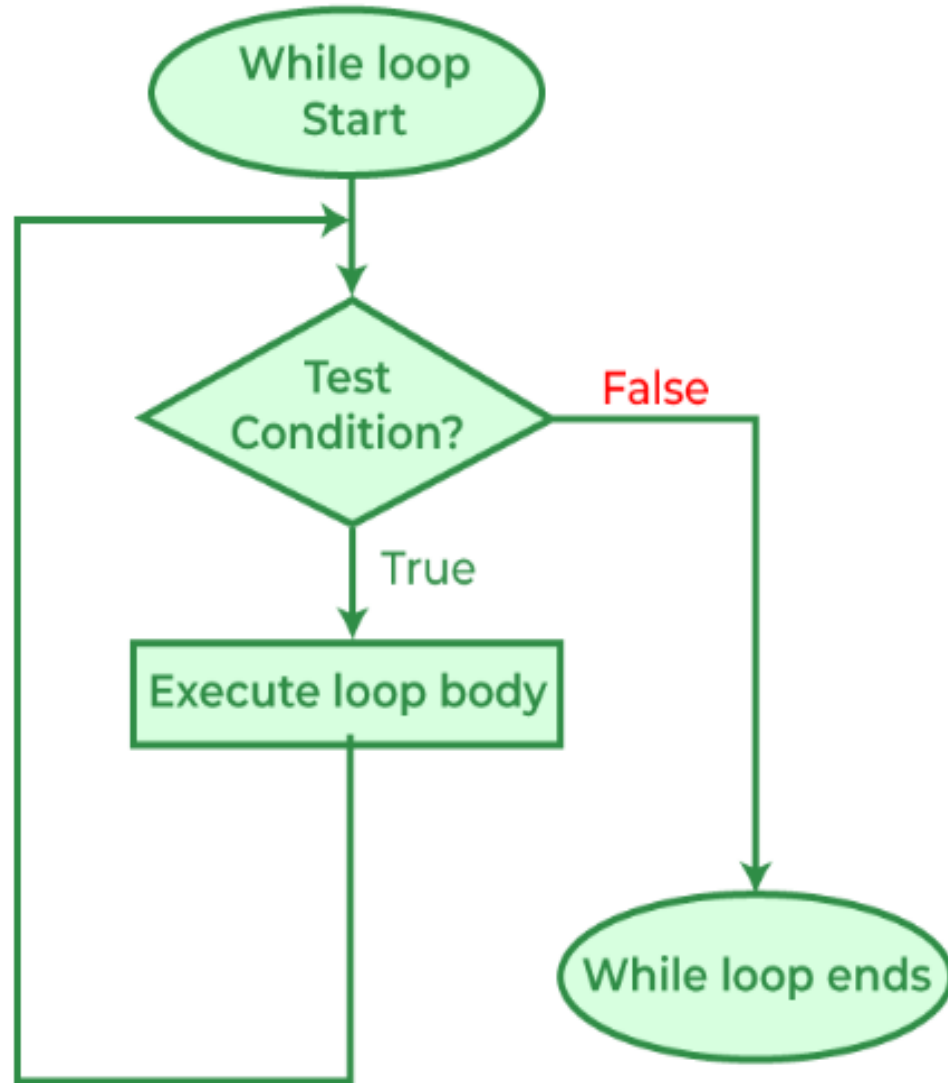
- ✓ The while statement, also called the *while loop*, *executes a block of statements as long as a specified condition* is true.
- ✓ The while statement has the following form:

while (condition)

statement

- ✓ *condition* is any C expression, and *statement* is a single or compound C statement.
- ✓ When program execution reaches a while statement, the following events occur:
 1. ***The condition is evaluated.***
 2. ***If condition evaluates to false (that is, zero), the while statement terminates, and execution*** passes to the first statement following *statement*.
 3. ***If condition evaluates to true (that is, nonzero), the C statement(s) in statement are executed.***
 4. Execution returns to step 1.

Flow Chart of the While loop



Example program of the While loop

```
1  /* Demonstrates a simple while statement */
2
3  #include <stdio.h>
4
5  int count;
6
7  int main()
8  {
9      /* Print the numbers 1 through 20 */
10
11     count = 1;
12
13     while (count <= 20)
14     {
15         printf("%d\n", count);
16         count++;
17     }
18     return 0;
19 }
20
```

- ✓ Because the while statement doesn't contain an initialization section, you must take care of initializing any variables before starting the while.
- ✓ In the while loop, incrementing is part of the body.
- ✓ What do you think would happen if you forgot to put the increment statement in this program?
- ✓ Your program wouldn't know when to stop, because count would always be 1, which is always less than 20.
- ✓ You might have noticed that a while statement is essentially a for statement without the initialization and increment components. Thus,
for (; condition ;)
is equivalent to
while (condition)
- ✓ Because of this equality, anything that can be done with a for statement can also be done with a while statement.

- ✓ When you use a while statement, any necessary initialization must first be performed in a separate statement, and the updating must be performed by a statement that is part of the while loop.
- ✓ When initialization and updating are required, most experienced C programmers prefer to use a for statement rather than a while statement.
- ✓ This preference is based primarily on source code readability.
- ✓ When you use a for statement, the initialization, test, and increment expressions are located together and are easy to find and modify.
- ✓ With a while statement, the initialization and update expressions are located separately and might be less obvious.
- ✓ A while statement is a C looping statement. It allows repeated execution of a statement or block of statements as long as the condition remains true (nonzero).
- ✓ If the condition is not true when the while command is first executed, the *statement(s) is never executed*.

The do...While Statement

- ✓ The third loop construct is the do...while loop, which executes a block of statements as long as a specified condition is true.
- ✓ The do...while loop tests the condition at the end of the loop rather than at the beginning, as is done by the for loop and the while loop.
- ✓ The structure of the do...while loop is as follows:

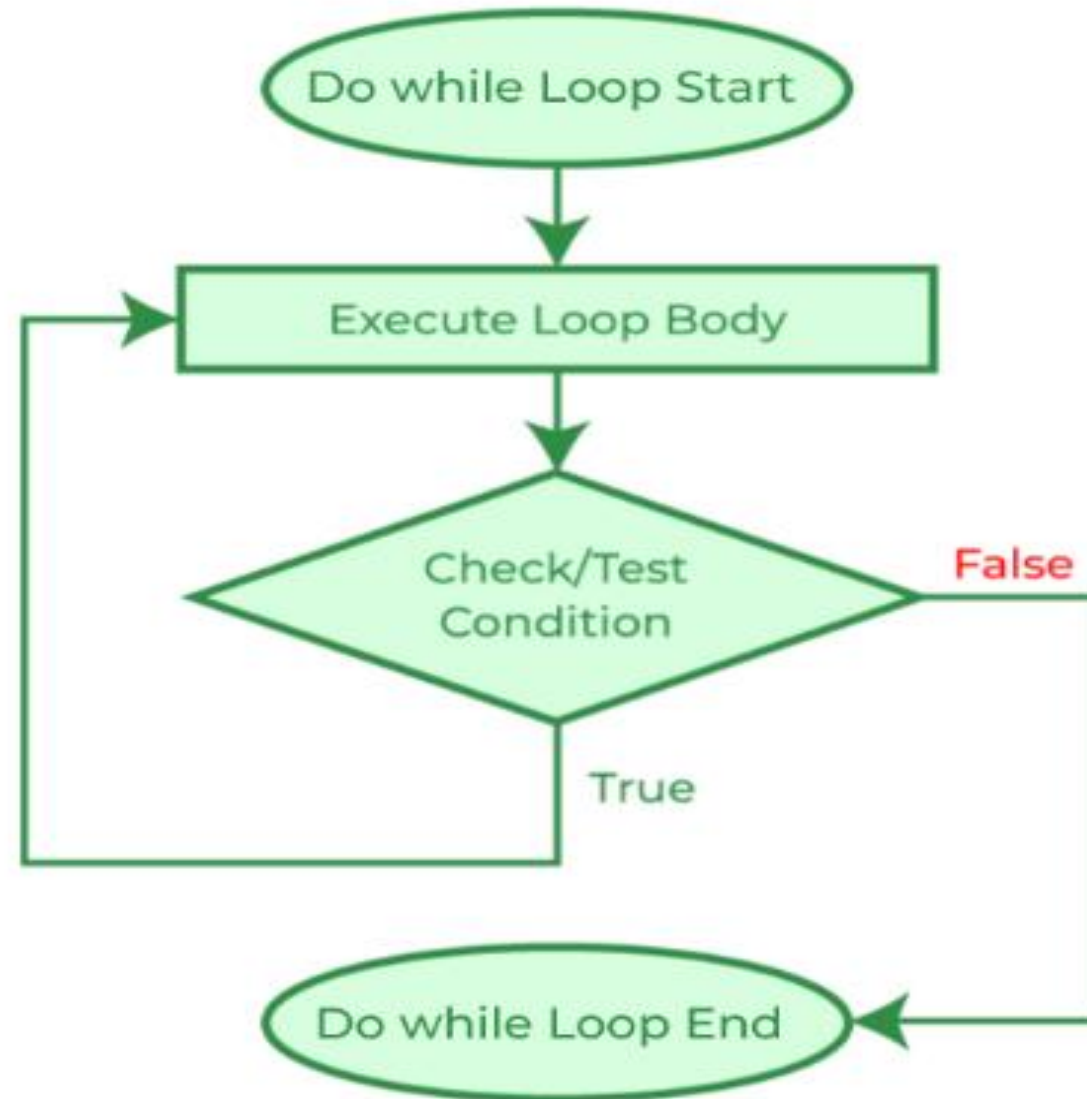
do

statement

while (*condition*);

- ✓ *condition* is any C expression, and *statement* is a single or compound C statement.
- ✓ When program execution reaches a do...while statement, the following events occur:
 1. **The statements in *statement* are executed.**
 2. ***Condition* is evaluated. If it's true, execution returns to step 1. If it's false, the loop terminates.**

Flow Chart of the do...While loop



- ✓ The statements associated with a do...while loop are always executed at least once.
- ✓ This is because the test condition is evaluated at the end, instead of the beginning, of the loop.
- ✓ In contrast, for loops and while loops evaluate the test condition at the start of the loop, so the associated statements are not executed at all if the test condition is initially false.
- ✓ The do...while loop is used less frequently than while and for loops.
- ✓ It is most appropriate when the statement(s) associated with the loop must be executed at least once.
- ✓ could, of course, accomplish the same thing with a while loop by making sure that the test condition is true when execution first reaches the loop.
- ✓ A do...while loop probably would be more straightforward, however.

Example program of the While loop

```
1  /* Demonstrates a simple do...while statement */
2
3  #include <stdio.h>
4
5  int get_menu_choice( void );
6
7  main()
8  {
9      int choice;
10
11     choice = get_menu_choice();
12
13     printf("You chose Menu Option %d\n", choice );
14
15     return 0;
16 }
17
18 int get_menu_choice( void )
19 {
20     int selection = 0;
21
22     do
23     {
24         printf("\n" );
25         printf("\n1 - Add a Record" );
26         printf("\n2 - Change a record");
27         printf("\n3 - Delete a record");
28         printf("\n4 - Quit");
29         printf("\n" );
30         printf("\nEnter a selection: " );
31
32         scanf("%d", &selection );
33
34     }while ( selection < 1 || selection > 4 );
35
36     return selection;
37 }
38
```

- ✓ The program provides a menu with four choices. The user selects one of the four choices, and then the program prints the number selected.
- ✓ Because you have to display a menu at least once to get an answer, it is appropriate to use a do...while loop.
- ✓ In the case of this program, the menu is displayed until a valid choice is entered.
- ✓ Line 34 contains the while part of the do...while statement and validates the value of the selection, appropriately named selection.
- ✓ If the value entered is not between 1 and 4, the menu is redisplayed, and the user is prompted for a new value.
- ✓ When a valid selection is entered, the program continues to line 36, which returns the value in the variable selection.
- ✓ A do...while statement is a C looping statement. It allows repeated execution of a statement or block of statements as long as the condition remains true (nonzero).
- ✓ Unlike the while statement, a do...while loop executes its statements at least once.

Summary

- ✓ C has three loop statements that control program execution: for, while, and do...while.
- ✓ Each of these constructs lets your program execute a block of statements zero times, one time, or more than one time, based on the condition of certain program variables.
- ✓ Many programming tasks are well-served by the repetitive execution allowed by these loop statements.
- ✓ Although all three can be used to accomplish the same task, each is different.
- ✓ The for statement lets you initialize, evaluate, and increment all in one command.
- ✓ The while statement operates as long as a condition is true.
- ✓ The do...while statement always executes its statements at least once and continues to execute them until a condition is false.
- ✓ Nesting is the placing of one command within another. C allows for the nesting of any of its commands.